File Edit Selection View Go Run Terminal Help run-integration-test.sh - Visual Studio Code

run-integration-test.sh

C: > Users > Satyam Kumar > Documents > Learning Journal > Udemy > Azure Databricks > run-integration-test.sh

```
JOB_ID=$(databricks jobs create --json '{   "name": "TestRun",
    "max_concurrent_runs": 1,
    "tasks": [
        {
            "task_key": "BatchTest",
            "run_if": "ALL_SUCCESS",
            "notebook_task": {
                "notebook_path": "/Shared/08-batch-test",
                "base_parameters": {
                    "Environment": "qa"
                },
                "source": "WORKSPACE"
            },
            "job_cluster_key": "BatchTest_cluster",
            "timeout_seconds": 0,
            "email_notifications": {}
        }
    ],
    "job_clusters": [
        {
            "job_cluster_key": "BatchTest_cluster",
            "new_cluster": {
                "spark_version": "12.2.x-scala2.12",
                "spark_conf": {
                    "spark.databricks.delta.preview.enabled": "true",
                    "spark.master": "local[*, 4]",
                    "spark.databricks.cluster.profile": "singleNode"
                },
                "azure_attributes": {
                    "first_on_demand": 1,
                    "availability": "ON_DEMAND_AZURE",
                    "spot_bid_max_price": -1
                }
```

1. Create a job to run the batch-test notebook
2. Trigger the job and get the run id.
3. Wait for the job to complete while it is running
4. Delete the job after completion
5. Publish the job result

Restricted Mode ⊗ 0 ⚠ 0

File Edit Selection View Go Run Terminal Help

run-integration-test.sh - Visual Studio Code

$ run-integration-test.sh ×

C: > Users > Satyam Kumar > Documents > Learning Journal > Udemy > Azure Databricks > $ run-integration-test.sh

```
        ],
        "job_clusters": [
            {
                "job_cluster_key": "BatchTest_cluster",
                "new_cluster": {
                    "spark_version": "12.2.x-scala2.12",
                    "spark_conf": {
                        "spark.databricks.delta.preview.enabled": "true",
                        "spark.master": "local[*, 4]",
                        "spark.databricks.cluster.profile": "singleNode"
                    },
                    "azure_attributes": {
                        "first_on_demand": 1,
                        "availability": "ON_DEMAND_AZURE",
                        "spot_bid_max_price": -1
                    },
                    "node_type_id": "Standard_DS3_v2",
                    "custom_tags": {
                        "ResourceClass": "SingleNode"
                    },
                    "spark_env_vars": {
                        "PYSPARK_PYTHON": "/databricks/python3/bin/python3"
                    },
                    "enable_elastic_disk": true,
                    "data_security_mode": "SINGLE_USER",
                    "runtime_engine": "STANDARD",
                    "num_workers": 0
                }
            }
        ],
        "format": "MULTI_TASK"
}'| jq '.job_id')
```

File Edit Selection View Go Run Terminal Help — run-integration-test.sh - Visual Studio Code

run-integration-test.sh

C: > Users > Satyam Kumar > Documents > Learning Journal > Udemy > Azure Databricks > run-integration-test.sh

```bash
    ],
    "format": "MULTI_TASK"
}'| jq '.job_id')

RUN_ID=$(databricks jobs run-now --job-id $JOB_ID | jq '.run_id')

job_status="PENDING"
  while [ $job_status = "RUNNING" ] || [ $job_status = "PENDING" ]
  do
    sleep 10
    job_status=$(databricks runs get --run-id $RUN_ID | jq -r '.state.life_cycle_state')
    echo Status $job_status
  done

  RESULT=$(databricks runs get-output --run-id $RUN_ID)
  databricks jobs delete --job-id $JOB_ID

  RESULT_STATE=$(echo $RESULT | jq -r '.metadata.state.result_state')
  RESULT_MESSAGE=$(echo $RESULT | jq -r '.metadata.state.state_message')
  if [ $RESULT_STATE = "FAILED" ]
  then
    echo "##vso[task.logissue type=error;]$RESULT_MESSAGE"
    echo "##vso[task.complete result=Failed;done=true;]$RESULT_MESSAGE"
  fi

  echo $RESULT | jq .
```